



تمرین صفر

برنامه‌سازی پیشرفته

طراح تمرین: علیرضا فرهی قهی

بهار ۱۴۰۵



۱. مقدمه و هدف تمرین

در این تمرین شما یک موتور پردازش متن ساده طراحی خواهید کرد. تمرکز اصلی روی استفاده صحیح از `std::vector`، `std::string`، پیاده‌سازی منطق‌های جستجو و کار با ساختارها (`struct`) است.

۲. صورت مسئله

شما باید برنامه‌ای بنویسید که یک متن خام (شامل علائم نگارشی و حروف بزرگ و کوچک) را دریافت کند. سپس لیستی از «کلمات ممنوعه» (Stop-words – لیستی از کلمات که باید در تحلیل نهایی نادیده گرفته شوند؛ این لیست می‌تواند شامل حروف اضافه یا هر کلمه خاصی باشد که طبق ورودی مسئله، نباید در آمار نهایی لحاظ گردد.) را دریافت کند. برنامه باید متن را پاکسازی کرده، کلمات ممنوعه را از آن حذف کند و در نهایت آمار کلمات باقی‌مانده را بر اساس بیشترین تکرار نمایش دهد.

۳. مشخصات ورودی

برنامه شما باید به گونه‌ای طراحی شود که بتواند ورودی‌ها را دقیقاً بر اساس قالب زیر مدیریت کند:

- متن اصلی:** متن ورودی می‌تواند شامل هر تعداد خط، اعم از خطوط پر و خالی باشد. کلمات با فاصله (Space)، تب (Tab) یا خط جدید (Newline) از هم جدا می‌شوند.
- شرط پایان متن:** پایان متن دقیقاً با عبارت `###END###` در یک خط مجزا مشخص می‌شود. تضمین می‌شود که قبل و بعد از این عبارت در آن خط، هیچ کاراکتر اضافه‌ای (حتی فاصله) وجود نداشته باشد.
- کلمات ممنوعه:** لیست کلمات ممنوعه دقیقاً در یک خط و بلافاصله پس از خط شامل عبارت `###END###` ارائه می‌شود. تضمین می‌شود که این کلمات تنها با یک فاصله از یکدیگر جدا شده‌اند و هیچ فاصله اضافی در ابتدا یا انتهای این خط وجود ندارد. (نکته: در صورتی که این خط کاملاً خالی باشد، به معنای آن است که هیچ کلمه‌ای نباید فیلتر شود).

مثال ورودی:

```
Hello world! This is a test.
The world is beautiful, but the test is hard.
###END###
is the a but this
```



۲. مشخصات خروجی

برنامه شما باید خروجی را دقیقاً با فرمت زیر (حساس به حروف بزرگ و کوچک و فاصله‌ها) در کنسول چاپ کند. هرگونه چاپ پیام‌های اضافه باعث کسر نمره خواهد شد.

1. **تعداد کل کلمات خام:** منظور تعداد کل کلماتی است که قبل از هرگونه پردازش، با فاصله یا خط جدید از هم جدا شده‌اند:

```
Total words original: <number>
```

2. **تعداد کلمات نهایی:** منظور تعداد کل کلمات معتبر باقی‌مانده پس از پاکسازی، حذف کلمات ممنوعه، حذف اعداد و حذف کلمات خالی است:

```
Total words after filter: <number>
```

3. **لیست تکرار کلمات:** ابتدا عبارت `Word Frequencies:` چاپ شده و سپس در هر خط، کلمه و تعداد تکرار آن چاپ می‌شود. لیست به صورت نزولی بر اساس تعداد تکرار مرتب شده باشد. در صورت برابری تعداد تکرار، مرتب‌سازی به صورت الفبایی (صعودی) انجام شود:

```
<word>: <count>
```

مثال خروجی (مطابق ورودی بالا):

```
Total words original: 15
Total words after filter: 7
Word Frequencies:
test: 2
world: 2
beautiful: 1
hard: 1
hello: 1
```



۵. الزامات پیاده‌سازی

برای کسب نمره در این تمرین، رعایت موارد زیر الزامی است:

1. **محدودیت در استفاده از کانتینرها (Containers) و ابزارها:** جهت پیاده‌سازی دقیق مفاهیم و مدیریت دستی داده‌ها، رعایت موارد زیر الزامی است:

۱. **ممنوعیت آرایه‌های C-Style:** استفاده از آرایه‌های ایستا با طول ثابت ممنوع است. برای ذخیره تمامی لیست‌های پویا، تنها مجاز به استفاده از `std::vector` هستید.

۲. **ممنوعیت کانتینرهای تناظری (Associative Containers):** استفاده از کانتینرهای `std::map`، `std::set` و موارد مشابه برای شمارش تعداد دفعات تکرار کلمات یا فیلتر کردن آن‌ها ممنوع است. (برای نگهداری هر کلمه و تعداد دفعات تکرار آن، می‌توان یک `struct` تعریف کرد. سپس با استفاده از یک `vector` از این `struct`، اطلاعات را مدیریت کرده و در نهایت آن را مرتب‌سازی کرد.)

۳. **ممنوعیت توابع آماده جستجو:** استفاده از توابع سطح بالای کتابخانه‌ی `algorithm` (مانند `std::count` یا `std::find`) برای پیدا کردن تکرارها مجاز نیست.

2. **پاکسازی (Sanitization):** فرآیند آماده‌سازی کلمات باید طبق قوانین زیر و به ترتیب ذکر شده اجرا شود:

۱. **یکسان‌سازی حروف:** تمام حروف کلمه (چه در متن اصلی و چه در لیست کلمات ممنوعه) باید به حروف کوچک (Lowercase) تبدیل شوند. به طوری که برای نمونه، تفاوتی میان `The`، `THE` و `the` وجود نداشته باشد.

۲. **حذف علائم از طرفین:** تنها علائم نگارشی شامل `! . , ' ; : "` باید از ابتدا و انتهای کلمه حذف شوند. علائم موجود در وسط کلمه (مانند آپوستروف در `don't`، خط تیره در `part-time` یا نقطه‌ی اعشار در `12.34`) باید در این مرحله حفظ شوند. توجه کنید که اگر کلمه‌ای تنها از علائم نگارشی تشکیل شده باشد، پس از این مرحله به رشته‌ای خالی تبدیل شده و باید نادیده گرفته شود.

۳. **فیلتر کردن اعداد (صحیح و اعشاری):** پس از پاکسازی علائم از طرفین، اگر کلمه‌ی باقی‌مانده یک عدد باشد، باید به طور کامل از متن حذف و نادیده گرفته شود. کلماتی عدد محسوب می‌شوند که:

- تماماً از ارقام (0-9) تشکیل شده باشند (مانند 1404).
- یا ارقام آن‌ها تنها با یک نقطه‌ی اعشار (.) از هم جدا شده باشد (مانند 12.34 یا 0.75).

(توضیح: کلماتی مثل 12.34.5 عدد معتبر نیستند و باید باقی بمانند).



۶. ساختار کد پیشنهادی

برای تمیزی کد، از شما انتظار می‌رود کل برنامه را در `main` ننویسید و منطق برنامه را به توابع مجزا بشکنید. پیاده‌سازی توابع زیر (یا مشابه آن‌ها) توصیه می‌شود:

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// ساختاری برای نگهداری کلمه و تعداد تکرار آن
struct WordCount {
    string word;
    int count;
};

// تابع تشخیص علائم نگارشی
bool isPunctuation(char c);

// تابع پاکسازی کلمه (حذف علائم نگارشی و تبدیل به حروف کوچک)
string sanitizeWord(string word);

// تابعی برای بررسی اینکه آیا یک کلمه در لیست کلمات ممنوعه هست یا خیر
bool isStopWord(string word, vector<string> stopWords);

// تابع جستجو و بروزرسانی شمارنده کلمات
vector<WordCount> updateWordCount(vector<WordCount> frequencies, string word);

// تابع مرتب‌سازی کلمات (بیشترین تکرار، سپس حروف الفبا)
vector<WordCount> sortWordCounts(vector<WordCount> frequencies);
```

موفق و سلامت باشید